



Artificial Intelligence

Mini Project 01: Escape The Maze

General Instructions

Due Date: 14 October 2026, 11:59 pm

- The project carries a total of **30 marks: Code (25 marks) and Report (5 marks)**.
- A **viva will be conducted** to assess your understanding of the implementation and the AI concepts used in the project.
- The project will be completed in **groups of 2**. Groups will be announced through GCR.
- You may discuss ideas and approaches with your classmates; however, **all submitted code must be your own work**. Submissions will be checked for similarity with other students' work and with solutions available online, including existing implementations of similar projects.
- If you have any questions or need clarification, feel free to **contact the TAs or instructor via email**.
- You can check your implementation by comparing your numbers with the table in **Checking your Work**.
- **There will be no extension to the deadline, so please start early and plan your work accordingly.**

Submission Instructions

- Submit a single zip file named `<group_number>_project1.zip` on **GCR** containing:
 - `search.py`, `heuristics.py`, `griever_hole.py`
 - `report.pdf`
- **Only one group member needs** to submit the .zip file.
- Before you submit, run your code on the mazes in *Checking your work* one last time, using exactly the files you are submitting. Code that does not run earns no points for the affected questions.

Project Background

This project is inspired by James Dashner's *The Maze Runner*. You wake up inside a rising metal box with no memory except your name. The doors open onto **the Glade**, a grassy square surrounded by enormous stone walls. Every morning, the walls open, revealing the entrance to **the Maze**. Every night, they close again, trapping anyone still inside with the deadly **Griever**s.

For the past three years, **the Runners** (Minho, Gally, Thomas) have entered the Maze every morning, exploring its corridors and manually mapping everything they discover. Despite their efforts, they have been unable to find the exit. You believe there is a better way. Instead of relying solely on manual exploration, you propose using search **algorithms** to systematically navigate the **Maze** and find a path from the **Glade** to the exit.

Project Overview

In this project, you will help the **Runners** escape. You will write general-purpose search algorithms and use them on **mazes with walls, thick ivy, and Griever territory**. Then you will formulate a harder problem yourself: collecting the pieces of the escape code before reaching the Griever Hole. You will be implementing classical AI search algorithms: DFS, BFS, DLS, IDS, UCS, Greedy best-first, and A*.

Files you will edit

File	What goes in it
<code>search.py</code>	Your search algorithms (Q1–Q7).
<code>heuristics.py</code>	Manhattan and Euclidean heuristics for the maze (Q6).
<code>griever_hole.py</code>	The key-hunt search problem and its heuristic (Q8–Q9).

Files you should read

File	What it contains
<code>problems.py</code>	The <code>SearchProblem</code> interface, the <code>MazeProblem</code> and <code>GraphProblem</code> classes, and the <code>mazeDistance</code> helper.
<code>util.py</code>	<code>Stack</code> , <code>Queue</code> , <code>PriorityQueue</code> , the <code>CUTOFF</code> value, <code>manhattanDistance</code> .
<code>maze.py</code>	Maze loading, terrain costs, and the neighbour order.

Files you can ignore

File	What it contains
<code>runner.py</code>	The command-line program you run to watch a search.
<code>display.py</code>	Text and graphical displays.
<code>mazes/</code>	Maze layouts. You may add your own.

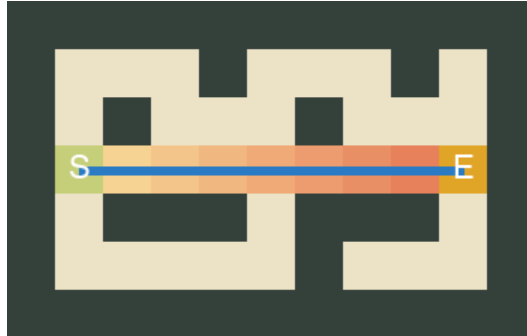
Welcome to the Glade

Download and unzip the project, open a terminal in the `maze_runner` folder, and run:

```
python runner.py
```

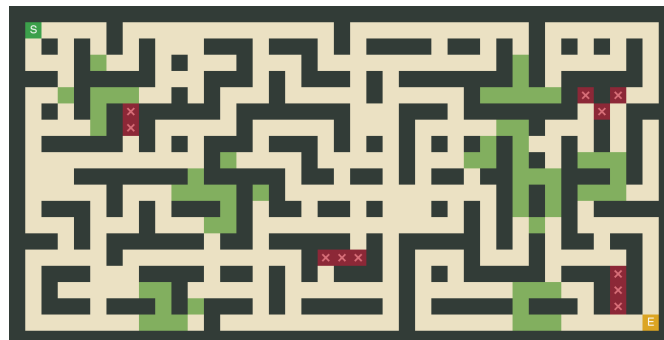
A window opens, and a runner walks through the tiny maze. At the moment, this fails with **Method not implemented: `breadthFirstSearch`**. That is your job. First, watch a runner whose only plan is to *always go right*:

```
python runner.py -l maze_tiny -a right      # escapes: the exit is straight ahead
python runner.py -l maze_small -a right    # stuck: a wall blocks the way east
```



maze_tiny: S is the Glade door, E is the exit. Always going right works here.

In the tiny maze, the exit happens to be straight ahead, so the plan works. In any real section of the maze (shown below), he walks into a wall after a few steps. Read **alwaysRightSearch** in **search.py**. It is a complete example of how an algorithm talks to a search problem, and it shows why we need algorithms that actually *search*.



ivy_maze: ivy (green) and Griever territory (red with a cross).

The Maze

Symbol	Meaning	Cost to enter
%	Wall (impassable)	—
space	Corridor	1
S	The Glade door (start)	1
E	The exit / Griever Hole	1
~	Thick ivy	3
G	Griever territory	10
K	A piece of the escape code (Q8–Q9)	1

Positions are (row, col) tuples, with row 0 at the top. The actions are 'North', 'South', 'East', and 'West'. A move costs the terrain cost of the cell you step *into*.

The search problem interface

Your algorithms never touch the maze directly. It only calls these four methods (see `problems.py`):

```
problem.getStartState()      # e.g. (1, 1)
problem.isGoalState(state)   # True / False
problem.getSuccessors(state) # [((1, 2), 'East', 1), ((2, 1), 'South', 3), ...]
problem.getCostOfActions(actions)
```

Running a search

```
python runner.py -l maze_small -a dfs          # -l maze, -a algorithm
python runner.py -l maze_small -a dls -d 40     # -d depth limit for DLS
python runner.py -l greedy_trap -a astar -H manhattan # -H heuristic
python runner.py -l keys_small -p keys -a bfs   # -p keys: the Q8 problem
python runner.py -l maze_medium -a bfs -t      # -t text output (no window)
python runner.py -l maze_medium -a bfs -q      # -q numbers only
python runner.py -l maze_medium --compare      # table of every algorithm
python runner.py --list                       # all mazes
```

In the window, the cells your algorithm **expanded** are shaded from light yellow (expanded early) to deep orange (expanded late), and the runner follows the path you returned. Press **Space** to skip the animation and **Q** to close. The terminal shows the path cost, the number of steps, the number of *nodes expanded* and the time taken.

Rules of the Maze

- Every algorithm returns a **list of actions** from the start to a goal ([]) if the start is already a goal). It returns **None** if there is no solution. DLS may also return **CUTOFF**.
- Calling `getSuccessors(state)` expands the state. Call it exactly once for each state you expand.
- Use the fringe containers in `util.py`. Push successors **in the order** `getSuccessors` returns them`.
- **Goal-test a state when you pop it** from the fringe, not when you generate it.
- Implement DFS, BFS, UCS, Greedy, and A* as **graph searches rather than tree searches**. Keep a set of expanded states and never expand a state twice. Mark a state as expanded when you pop and expand it, not when you push it.

Hint: one algorithm, many fringes

DFS, BFS, UCS, Greedy, and A* are the same graph-search loop. They differ only in the kind of fringe and the priority used to order it. Once your DFS works, BFS should take you a few minutes. Consider writing the loop once as a helper function.

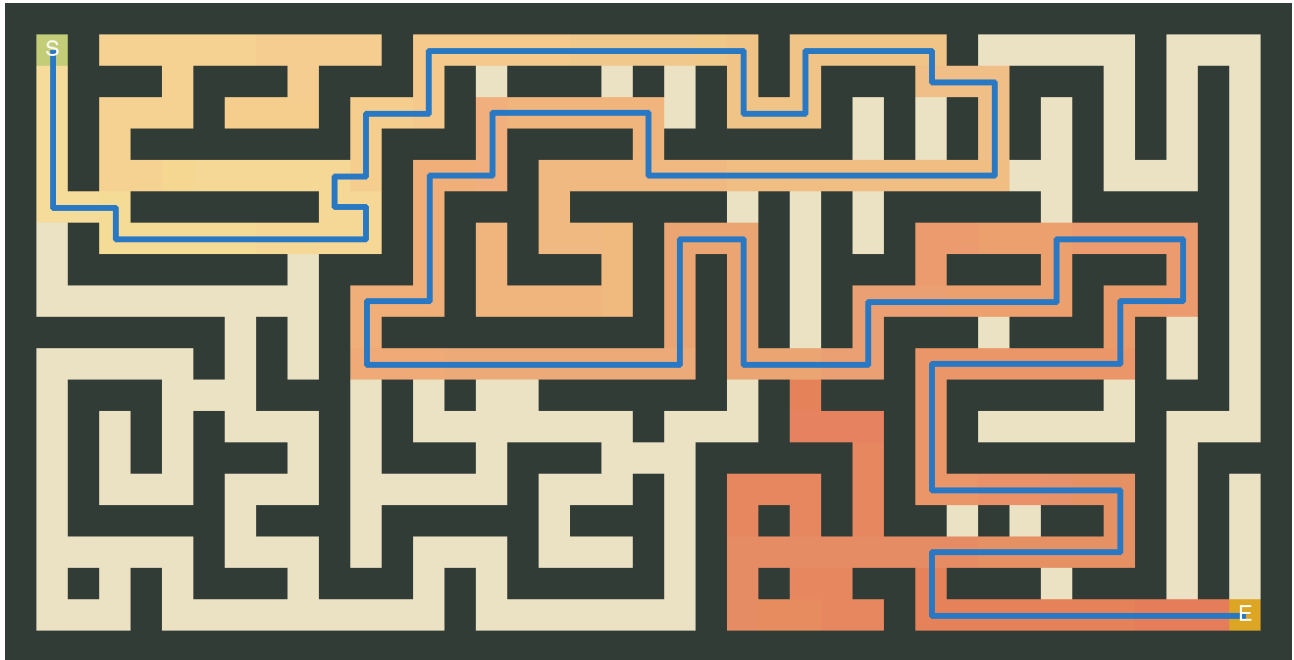
```
fringe ← container holding the start node
expanded ← empty set
loop:
  if fringe is empty: return failure
  node ← fringe.pop()
  if node.state is a goal: return node's actions
  if node.state not in expanded:
    add node.state to expanded
    for each (successor, action, cost) in getSuccessors(node.state):
      push a child node onto the fringe
```

Q1 · Depth-First Search (2 points)

On your first day, you ignore Minho's advice and run blindly: at every junction you take the first corridor and keep going until you hit a dead end, then back up.

Implement `depthFirstSearch` in `search.py`. Your code should solve these quickly:

```
python runner.py -l maze_tiny -a dfs
python runner.py -l maze_small -a dfs
python runner.py -l maze_medium -a dfs
```



DFS on maze_medium

Hint

A fringe entry can hold more than a state. Pushing `(state, actionsSoFar)` is the simplest way to remember how you reached each state.

Q2 · Breadth-First Search (2 points)

Minho spreads the Runners' maps across the table in the Map Room. "We don't run blind. We check every corridor one step away, then every corridor two steps away..."

Implement `breadthFirstSearch`. It must return a path with the **fewest steps**.

```
python runner.py -l maze_medium -a bfs
python runner.py -l open_glade -a bfs
```

Your BFS must not depend on the problem being a maze: in Q8 you will run it, unchanged, on a problem whose states are not positions.

Q3 · Depth-Limited Search (3 points)

The doors close at sunset. A Runner who is not back in the Glade by then does not survive the night, so nobody runs further than they can get back from.

Implement `depthLimitedSearch(problem, limit)`: depth-first search that never goes deeper than `limit` actions. It returns one of three things:

Return value	Meaning
a list of at most <code>limit</code> actions	found a goal
<code>CUTOFF</code> (from <code>util</code>)	no goal found, but the depth limit stopped the search somewhere, so a deeper solution may exist
<code>None</code>	no goal found and the limit was never reached: there is no solution at all

The start state has depth 0. A state at depth `limit` may be goal-tested, but it is not expanded. Compare with `result is CUTOFF`.

The explored-set trap

Do *not* use a global explored set here. Take the graph `A→B, A→C, B→C, C→D, D→G` with limit 3. DFS reaches C first through B, at depth 2, so D is at depth 3 and gets cut off. If C is now "explored", the route `A→C→D→G`, which fits the limit exactly, is never tried. Instead, avoid cycles by skipping any successor that is **already on the current path**. A recursive implementation is the most natural.

```
python runner.py -l maze_small -a dls -d 60      # escapes
python runner.py -l maze_small -a dls -d 30      # CUTOFF: the doors close first
python runner.py -l maze_tiny -a dls -d 8
```

Q4 · Iterative Deepening Search (2 points)

Newbies are not allowed to run far. On day one they go one corridor out and back, on day two two corridors, and so on, until one day the exit is within reach.

Implement `iterativeDeepeningSearch` using your `depthLimitedSearch` with limits 0, 1, 2, ... Stop as soon as the result is not `CUTOFF`: either a solution, or `None`, which proves there is no solution at all.

```
python runner.py -l maze_small -a ids
python runner.py -l maze_medium -a ids -q
```

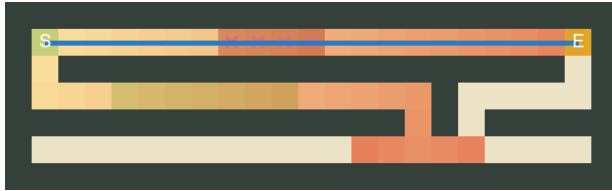
IDS returns a path with the fewest steps, like BFS, but look at the number of nodes expanded. Now try it on the `open_glade` maze. The runner gives up after 60 seconds (`--timeout`). You will explain what happens in your report.

Q5 · Uniform-Cost Search (3 points)

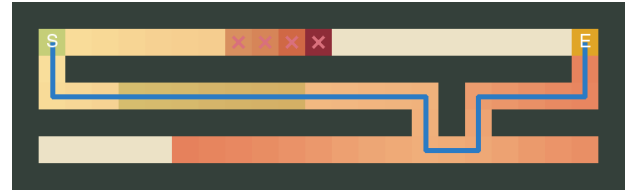
Some corridors are choked with ivy that takes three times as long to push through. Others are Griever territory: technically passable, but you will pay for it.

Implement `uniformCostSearch`, which expands the node with the lowest path cost $g(n)$ first and returns the **cheapest** path. Compare it with BFS on Griever Alley:

```
python runner.py -l griever_alley -a bfs      # 20 steps, cost 56 (straight through the
Griever's)
python runner.py -l griever_alley -a ucs      # 28 steps, cost 42
python runner.py -l ivy_maze -a ucs
```



BFS: fewest steps, cost 56.



UCS: cheapest path, cost 42.

Hint

`util.PriorityQueue` pops the item with the lowest priority and breaks ties first-in-first-out. You may push the same state more than once and skip it when it is popped a second time, or use `update()`.

Q6 · Greedy Best-First Search (3 points)

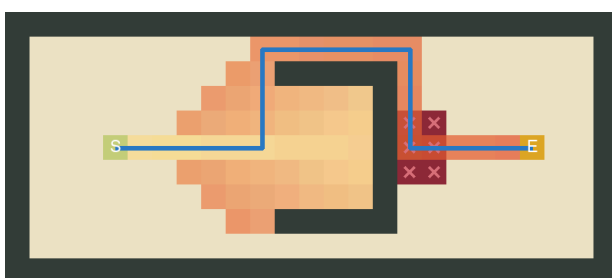
Gally has a simpler plan: "We know roughly where the exit is. Just run towards it."

First implement two heuristics in `heuristics.py`. Both take a `(row, col)` state and the problem (the goal is `problem.goal`):

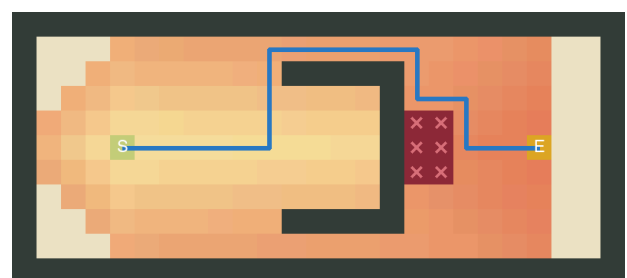
- `manhattanHeuristic`: $|\Delta\text{row}| + |\Delta\text{col}|$
- `euclideanHeuristic`: the straight-line distance

Then implement `greedyBestFirstSearch(problem, heuristic)` in `search.py`. It always expands the node that *looks* closest to the goal, the one with the lowest $h(n)$.

```
python runner.py -l greedy_trap -a gbfs -H manhattan
python runner.py -l open_glade -a gbfs -H euclidean
```



Greedy on `greedy_trap`: 60 nodes expanded, but the path costs 52.



A* on the same maze: cost 25.

Q7 · A* Search (3 points)

Thomas combines both ideas: Minho's careful bookkeeping of how far each Runner has already gone, and Gally's sense of where the exit is.

Implement `aStarSearch(problem, heuristic)`, which expands the node with the lowest $f(n) = g(n) + h(n)$. The Manhattan distance never overestimates, because every move costs at least 1, so A* with it must return the cheapest path while expanding far fewer nodes than UCS.

```
python runner.py -l maze_big -a ucs -q
python runner.py -l maze_big -a astar -H manhattan -q
python runner.py -l ivy_maze --compare
```

Q8 · The Griever Hole (3 points)

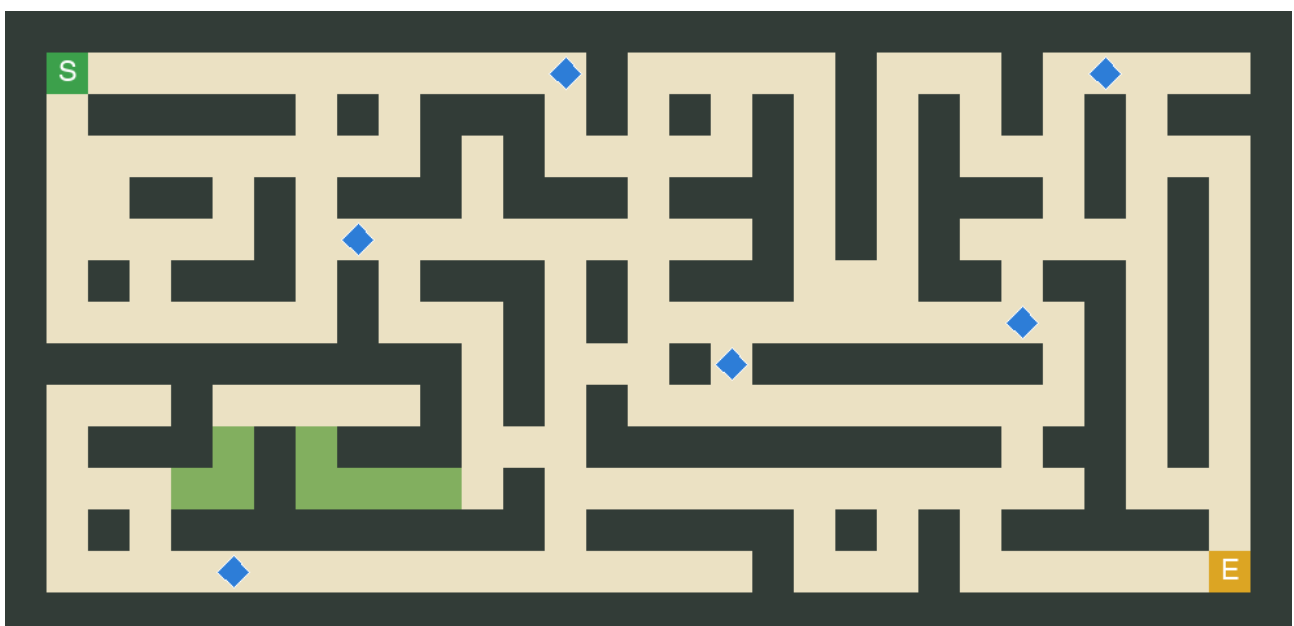
*The exit is not a door. It is the Griever Hole, and it only opens when the escape code is entered: six words hidden in the Maze. **FLOAT. CATCH. BLEED. DEATH. STRONG. PUSH.***

The code pieces (**k**) are scattered through the corridors. Thomas must step on **every** piece, in any order, and then reach the exit **E**. Complete `KeyHuntProblem` in `griever_hole.py`: `getStartState`, `isGoalState` and `getSuccessors`.

This question tests your **problem formulation**. You choose the state representation. It must be:

- **hashable**: tuples, frozensets, and ints, not lists, sets, or dicts;
- **a tuple whose first element is Thomas's `(row, col)`**;
- **small**: only what you need to decide what happens next. Ask yourself: does the *order* in which the pieces were collected matter for the future?

```
python runner.py -l keys_tiny -p keys -a bfs      # 26 steps
python runner.py -l keys_small -p keys -a bfs    # 54 steps
```



`keys_medium`: six code pieces (blue diamonds) to collect before the exit.

Q9 · Race to the Griever Hole (4 points)

The Grievers are coming. There is no time to search every corridor.

Write `keyHuntHeuristic(state, problem)` in `griever_hole.py`. The grader runs A* with it on `keys_medium`:

```
python runner.py -l keys_medium -p keys -a astar -H keyhunt
python runner.py -l keys_medium -p keys -a ucs -q      # 9695 expansions to beat
```

Your heuristic must be **consistent**: for every state s and every successor s' reached with step cost c , $h(s) \leq c + h(s')$, and $h = 0$ at goal states. The grader checks this on the states A* visits. An inconsistent or trivial heuristic earns **0 points**.

Nodes expanded on <code>keys_medium</code>	Points
consistent and non-trivial ($h > 0$ at the start)	1
at most 7000	2
at most 2600	3
at most 1200	4

Tools and tips

`mazeDistance(pos1, pos2, problem.maze, problem.heuristicInfo)` returns the true number of steps between two cells, and caches the results in the dictionary you pass. Start simple: what is the least Thomas still has to walk if only *one* piece remained? Then ask how to account for *all* of them without ever overestimating. Always take the `max` of admissible estimates, never their sum, unless you can prove the sum is admissible.

Checking your work

A correct implementation that follows the *Rules of the Maze* gives these results. Use `-q` to print only the numbers, e.g. `python runner.py -l maze_medium -a bfs -q`. Greedy and A* use `-H manhattan` (the default).

Q	Command	Path cost	Steps	Nodes expanded
1	<code>-l maze_small -a dfs</code>	50	50	82
1	<code>-l maze_medium -a dfs</code>	162	162	233
2	<code>-l maze_tiny -a bfs</code>	8	8	27
2	<code>-l maze_medium -a bfs</code>	66	66	407
2	<code>-l open_glade -a bfs</code>	32	32	344
3	<code>-l maze_small -a dls -d 60</code>	50	50	52
3	<code>-l maze_small -a dls -d 30</code>	CUTOFF	—	32
4	<code>-l maze_small -a ids</code>	50	50	1352
5	<code>-l griever_alley -a ucs</code>	42	28	50
5	<code>-l ivy_maze -a ucs</code>	70	64	418
6	<code>-l greedy_trap -a gbfs</code>	52	25	60
6	<code>-l maze_medium -a gbfs</code>	80	80	81
7	<code>-l greedy_trap -a astar</code>	25	25	155
7	<code>-l maze_big -a astar</code>	124	124	679

8	-l keys_tiny -p keys -a bfs	26	26	490
8	-l keys_small -p keys -a bfs	54	54	1723

Path cost and steps must match exactly for BFS, IDS, UCS and A*, which are optimal. For DFS and greedy search, any legal path is correct, but with the conventions you should get the numbers shown. **Nodes expanded** can differ slightly if you push successors in reverse order or, for BFS, goal-test when generating. That is allowed. A large difference means something is wrong.

Report (5 points)

Submit a short, well-formatted report in PDF format. **Do not include screenshots of code snippets in the report. Including code screenshots will result in 0 marks for the report.** Use the `--compare` option to collect your numbers.

1. **AI Assistant Used:** Clearly state the AI assistant/tool you used and briefly describe the purpose for which you used it.
2. **Results table.** Run `python runner.py -l <maze> --compare` on `maze_medium`, `open_glade`, and `ivy_maze` and include the tables.
3. **DFS vs BFS.** Why is the DFS path on `maze_medium` so long? Which of the two is complete and which is optimal, and when?
4. **IDS.** IDS is fast on `maze_small` but hopeless on `open_glade`, even though both mazes are small. Explain why, using the structure of the two mazes and how your DLS avoids cycles. What does IDS gain over BFS in exchange?
5. **Costs.** On `griever_alley`, why do BFS and UCS disagree? On which kinds of mazes are they guaranteed to return equally good paths?
6. **Heuristics.** Explain what goes wrong for greedy search on `greedy_trap`. Is the Euclidean heuristic admissible here? Compare the nodes expanded by A* with Manhattan and with Euclidean on `maze_big`, and explain the difference.
7. **Your Q9 heuristic.** Describe it, argue why it is consistent, and report the nodes it expands.

Troubleshooting

The window doesn't open / "no display" / tkinter is missing

Use text output: `python runner.py -l maze_small -a bfs -t`. Everything else works without graphics, and so does the grader. On Linux, `sudo apt install python3-tk` adds tkinter.

RecursionError in DLS / IDS

The runner (and the grader) raise Python's recursion limit to 20,000, which is enough for every provided maze. If you want to avoid recursion entirely, use an explicit `Stack` that stores the current path with each entry.

My numbers don't match the table

Check the *Rules of the Maze*: goal-test on pop, push successors in the given order, mark states as expanded when popped, and call `getSuccessors` only when expanding. Then test on a tiny graph where you can work out the right answer by hand (see *Checking your work*).

My search hangs

You are probably re-expanding states: check your explored set. The runner stops after 60 seconds (`--timeout`).